```
  it 'should not include :fish_oil' do
    expect(CookieRecipe.new.ingredients).not_to include(:fish_oil)
  end
end
```

These appear in the output when you run your specs with the documentation formatter:

```
$ rspec spec/cookie_recipe_spec.rb --format documentation

CookieRecipe#ingredients
  should include :butter, :milk and :eggs
  should not include :fish_oil

Finished in 0.00197 seconds (files took 0.08433 seconds to load)
2 examples, 0 failures
```

Notice the duplication in the specs, though. We've explained each example twice: once in the description, and once in the code. If we remove the former, RSpec will write its own description based on the code:

10-exploring-rspec-expectations/06/spec/cookie_recipe_no_doc_strings_spec.rb

```
RSpec.describe CookieRecipe, '#ingredients' do
  specify do
    expect(CookieRecipe.new.ingredients).to include(:butter, :milk, :eggs)
  end

  specify do
    expect(CookieRecipe.new.ingredients).not_to include(:fish_oil)
  end
end
```

We've also switched to the specify alias to avoid the grammatically awkward phrase it expect. When we run this version of the specs:

```
$ rspec spec/cookie_recipe_no_doc_strings_spec.rb --format documentation

CookieRecipe#ingredients
  should include :butter, :milk, and :eggs
  should not include :fish_oil

Finished in 0.00212 seconds (files took 0.08655 seconds to load)
2 examples, 0 failures
```

...the output is exactly the same as when we wrote the descriptions by hand. This way, though, the specs are a litttle more future-proof. If we want to change an example later—to make the recipe dairy-free, for instance—we don't have to worry about keeping our English description in sync with the code.

To generate these descriptions, RSpec takes the description of the last-executed expectation and prefixes it with should or should not.